

OS Lab

Skill-10

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_INPUT 1024

void show_current_directory()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("getcwd");
        return;
    }

    printf("Current directory: %s\n", cwd);
}

void save_state()
{
    char cwd[PATH_MAX];
    FILE *file;

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("getcwd");
        return;
    }
}
```

```

file = fopen("shell_state.txt", "w");

if (file == NULL)
{
    perror("fopen");
    return;
}

fprintf(file, "Last working directory: %s\n", cwd);

fclose(file);

printf("Shell state saved.\n");
}

void cleanup()
{
    printf("Cleaning up resources...\n");

    /*
     * Add any dynamically allocated memory,
     * files, or other resources here.
     */

    printf("Cleanup completed.\n");
}

int main()
{
    char input[MAX_INPUT];

    printf("Shell State Management\n");
    printf("Commands: pwd, exit\n\n");

    while (1)
    {
        printf("state-shell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';
    }
}

```

```
if (strcmp(input, "pwd") == 0)
{
    show_current_directory();
}
else if (strcmp(input, "exit") == 0)
{
    printf("Exit request received.\n");

    save_state();
    cleanup();

    printf("Shell exiting...\n");
    break;
}
else if (strlen(input) == 0)
{
    continue;
}
else
{
    printf("Invalid command: %s\n", input);
}
}

return 0;
}
```

```

lahari@DESKTOP-AIC33F7:~$ cd ~/OSLab/shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ cd ~/OSLab/shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ nano shell_state.c
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ gcc shell_state.c -o shell_state
shell_state.c:7: warning: "MAX_INPUT" redefined
    7 | #define MAX_INPUT 1024
      |
In file included from /usr/include/x86_64-linux-gnu/bits/local_lim.h:38,
                 from /usr/include/x86_64-linux-gnu/bits/posix1_lim.h:161,
                 from /usr/include/limits.h:195,
                 from /usr/lib/gcc/x86_64-linux-gnu/13/include/limits.h:205,
                 from /usr/lib/gcc/x86_64-linux-gnu/13/include/syslimits.h:7,
                 from /usr/lib/gcc/x86_64-linux-gnu/13/include/limits.h:34,
                 from shell_state.c:5:
/usr/include/linux/limits.h:11: note: this is the location of the previous definition
   11 | #define MAX_INPUT      255      /* size of the type-ahead buffer */
      |
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./shell_state
Shell State Management
Commands: pwd, exit

state-shell> pwd
Current directory: /home/lahari/OSLab/shell_lab
state-shell> exit
Exit request received.
Shell state saved.
Cleaning up resources...
Cleanup completed.
Shell exiting...
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ cat shell_state.txt
Last working directory: /home/lahari/OSLab/shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ export NAME=Lahari
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ echo $NAME
Lahari
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ |

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

```

```

#define MAX_INPUT 1024

```

```

int valid_variable_name(const char *name)
{
    int i;

    if (name == NULL || name[0] == '\0')
        return 0;

    /* First character must be letter or underscore */
    if (!isalpha(name[0]) || name[0] == '_')
        return 0;

```

```

/* Remaining characters */
for (i = 1; name[i] != '\0'; i++)
{
    if (!(isalnum(name[i]) || name[i] == '_'))
        return 0;
}

return 1;
}

```

```

void handle_export(char *command)
{

```

```

    char *assignment;
    char *name;
    char *value;

```

```

/*
 * Find '='
 *
 * Example:
 * export NAME=Lahari
 *
 * NAME = variable name
 * Lahari = value
 */

```

```

assignment = strchr(command, '=');

```

```

if (assignment == NULL)
{
    printf("Invalid export syntax.\n");
    printf("Use: export NAME=value\n");
    return;
}

```

```

/* Separate name and value */
*assignment = '\0';

```

```

name = command;
value = assignment + 1;

```

```

/* Validate variable name */
if (!valid_variable_name(name))

```

```

{
    printf("Invalid variable name: %s\n", name);
    return;
}

/*
 * setenv()
 *
 * 1 = overwrite existing variable
 */
if (setenv(name, value, 1) != 0)
{
    perror("setenv");
    return;
}

printf("Exported: %s=%s\n", name, value);
}

void show_variable(char *name)
{
    char *value = getenv(name);

    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("Variable '%s' is not defined.\n", name);
}

int main()
{
    char input[MAX_INPUT];

    printf("Environment Variable Shell\n");
    printf("Commands:\n");
    printf(" export NAME=value\n");
    printf(" show NAME\n");
    printf(" exit\n\n");

    while (1)
    {
        printf("export-shell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)

```

```

        break;

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
{
    printf("Exiting shell...\n");
    break;
}

if (strncmp(input, "export ", 7) == 0)
{
    char *command = input + 7;

    while (*command == ' ')
        command++;

    handle_export(command);
}
else if (strncmp(input, "show ", 5) == 0)
{
    char *name = input + 5;

    while (*name == ' ')
        name++;

    show_variable(name);
}
else if (strlen(input) == 0)
{
    continue;
}
else
{
    printf("Invalid command.\n");
}
}

return 0;
}

```

```
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ cd ~/OSLab/shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ nano export_shell.c
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ gcc export_shell.c -o export_shell
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./export_shell
Environment Variable Shell
Commands:
    export NAME=value
    show NAME
    exit

export-shell> export NAME=Lahari
Exported: NAME=Lahari
export-shell> show NAME
NAME=Lahari
export-shell> export NAME=Student
Exported: NAME=Student
export-shell> show NAME
NAME=Student
export-shell> export MY-NAME=hello
Invalid variable name: MY-NAME
export-shell> export MY_NAME=hello
Exported: MY_NAME=hello
export-shell> |
```